

Avaliação e Mitigação de Canais Encobertos Baseados em Estados de Substituição LRU e RRIP

Adriano Araújo¹, Artur Rizzi¹, Eduardo Aniceto¹, Izabel Chaves¹ ,
Pedro Magalhães¹

¹ Departamento de Ciência da Computação – Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-610 – Belo Horizonte – Brasil

{iopchaves, 1464597, 1568608, 1593221, 1447017}@pucminas.edu.br

1. Introdução

O desempenho das arquiteturas de computadores modernas é altamente impactado pela hierarquia de memória, principalmente no nível das memórias cache, como a L1 (*Level 1*), que é a mais próxima fisicamente da CPU, e também a LLC (*Last-Level Cache* ou Cache de Último Nível). A utilização de caches reduz a latência de acesso à memória principal, armazenando temporariamente dados que são acessados com frequência. Entretanto, como o espaço disponível nas caches é relativamente pequeno, é necessário utilizar políticas de substituição para decidir quais blocos devem ser removidos quando um novo bloco precisa ser armazenado. Esse tipo de mecanismo também está presente em outros sistemas que possuem recursos limitados, como sistemas de gerenciamento de memória e bancos de dados, nos quais é necessário decidir quais informações devem permanecer armazenadas e quais podem ser substituídas. Entre as políticas utilizadas em caches, é destacado a LRU (*Least-Recently Used*, ou Menos Recentemente Usada) e suas variações computacionalmente mais baratas, como a PLRU (*Pseudo-LRU*).

No entanto, essas políticas podem introduzir vulnerabilidades de segurança, permitindo ataques de canais laterais (*side-channels*) e canais encobertos (*covert-channels*). Ataques como *Flush+Reload* exigem uma falha de cache (*cache miss*) para alterar o estado da memória. Na política LRU, porém, a própria atualização dos metadados relacionados à idade dos blocos pode vaziar informações, permitindo a realização de ataques mesmo quando ocorre um acerto de cache (*cache hit*). Isso possibilita que um atacante consiga inferir o padrão de acesso da vítima apenas medindo o tempo de suas próprias operações, tornando o ataque mais furtivo, mais rápido e capaz de contornar defesas baseadas no monitoramento de falhas de cache [Xiong et al. 2021]. Os processadores comerciais, a exemplo da Intel e AMD, utilizam arquiteturas de "caches seguras", que depende de particionamento ou preenchimento aleatório visando mitigar a vulnerabilidade, mas não solucionam totalmente. Como solução definitiva para essas arquiteturas, o isolamento não deve abranger apenas os dados, mas os metadados LRU também devem ser devidamente bloqueados ou particionados.

Neste contexto, a motivação desta pesquisa surge da lacuna existente na avaliação empírica e quantitativa das contramedidas propostas na literatura. Trabalhos anteriores analisaram a utilização de defesas para reduzir ataques à hierarquia de cache, enquanto estudos sobre os estados da política LRU demonstraram que esses estados podem ser explorados para estabelecer canais encobertos [Xiong et al. 2021, Xiong and Szefer 2020].

No entanto, ainda falta uma avaliação rigorosa da banda residual desses canais quando mecanismos de proteção estão presentes. Além disso, ataques à LLC já demonstraram ser práticos em processadores modernos [Liu et al. 2015], reforçando a necessidade de investigar o comportamento desses canais em diferentes níveis da hierarquia de memória. Essa questão se torna ainda mais relevante diante do uso de políticas modernas de substituição na LLC, como a família RRIP, que utiliza estados de predição para determinar quais blocos devem ser substituídos [Jaleel et al. 2010].

A hipótese que direciona este trabalho é que as camadas inferiores da hierarquia de memória, especificamente a LLC equipada com políticas de substituição modernas, não são totalmente imunes a ataques que exploram metadados de substituição. Além disso, isolar apenas os dados na cache é insuficiente, sendo necessário também um tratamento seguro e particionado dos estados da política para mitigar os vazamentos sem comprometer de forma significativa o desempenho.

Dessa forma, o objetivo geral deste estudo é avaliar, por meio do simulador *gem5*, o comportamento de canais encobertos baseados em políticas de substituição de cache, comparando as políticas LRU e RRIP e analisando seu impacto na segurança e no desempenho do sistema.

Para alcançar este propósito, são definidos os seguintes objetivos específicos:

- Implementar e configurar no simulador *gem5* as políticas de substituição LRU e RRIP para a avaliação dos canais encobertos.
- Implementar um canal encoberto baseado no estado da política de substituição e avaliar sua capacidade de transmitir informações.
- Comparar o comportamento das políticas LRU e RRIP utilizando métricas como taxa de transmissão e taxa de erro.
- Avaliar o impacto das políticas de substituição no desempenho do sistema por meio de métricas como IPC e número de acessos à cache.
- Avaliar uma estratégia simples de mitigação baseada no isolamento ou particionamento do estado de substituição e comparar seus resultados com as configurações sem mitigação.

2. Revisão e Análise Crítica

O artigo *Leaking Information Through Cache LRU States in Commercial Processors and Secure Caches* [Xiong et al. 2021] investiga como os estados internos das políticas de substituição LRU podem ser explorados para criar canais laterais e canais encobertos. A principal contribuição do trabalho está em demonstrar que um acesso que resulta apenas em um *cache hit* ainda é capaz de modificar o estado de substituição da cache. Dessa forma, um processo receptor pode induzir uma substituição e utilizar a diferença no tempo de acesso para inferir se outro processo acessou determinada linha de cache, possibilitando o vazamento de informações sem que o transmissor necessariamente provoque um *cache miss*.

A avaliação experimental constitui um dos principais pontos fortes do artigo, pois os autores demonstram a viabilidade do ataque em processadores comerciais Intel e AMD, considerando cenários de compartilhamento por *hyper-threading* e *time-slicing*. Os resultados mostram taxas de transmissão significativamente maiores no cenário de *hyper-threading*, chegando a aproximadamente 500 Kbps em processadores Intel, enquanto no

time-slicing as taxas são drasticamente reduzidas. Essa diferença evidencia uma limitação importante: embora o canal baseado em LRU seja viável, sua eficiência depende diretamente da arquitetura do processador e da forma como os processos compartilham os recursos da CPU.

Além da avaliação em processadores reais, o artigo analisa a vulnerabilidade de propostas de caches seguras, como a *Partition-Locked Cache* e a *Random Fill Cache*, utilizando o simulador gem5. Os resultados indicam que proteger ou particionar apenas os dados armazenados na cache não é suficiente quando os metadados responsáveis pela política de substituição continuam compartilhados. Entretanto, uma limitação do trabalho é que essa avaliação das caches seguras foi realizada em ambiente simulado, enquanto a validação experimental ocorreu apenas para processadores comerciais convencionais. Assim, o artigo evidencia uma lacuna relevante, que é a necessidade de investigar mecanismos de isolamento que protejam não apenas as linhas de dados, mas também os estados de substituição, avaliando de forma simultânea sua capacidade de mitigar o vazamento e seu impacto no desempenho da hierarquia de memória.

3. Trabalhos Relacionados

3.1. Canais e Estados de Substituição em Caches

A pesquisa “*Last-Level Cache Side-Channel Attacks are Practical*”[Liu et al. 2015], demonstrou o quão viável é explorar caches compartilhados, como a LLC - Last-Level Cache, em ambientes multi threaded e de nuvem para extrair chaves criptográficas. No entanto, enquanto os ataques clássicos focam no tempo do acesso à memória, o artigo em questão refina essa abordagem ao demonstrar que a própria política de substituição LRU (Least Recently Used) retém estado e é capaz do vazamento de informação. Como limitação, o ataque depende do uso de *large pages* pelo atacante para montar os conjuntos de desvio (*eviction sets*), e os próprios autores reconhecem que desabilitar esse recurso na VMM seria a defesa mais simples. Além disso, a arquitetura de LLC dividida em *slices* obriga os autores a montar esses conjuntos por tentativa e erro. O ataque também exige etapa manual de análise e os autores admitem não ter testado o método em ambiente ruidoso ou em nuvem real.

Ademais, a análise dos componentes e estados internos do processador se conecta com “*Are Coherence Protocol States Vulnerable to Information Leakage?*”[Yao et al. 2018], porque ambas as pesquisas compartilham a mesma ideia de que estruturas de otimização de hardware são capazes de vazarem dados através de suas alterações de estado, não se limitando ao tempo de transferência de dados. Contudo, os autores mostram que, acima de 500 Kbps, a maioria das configurações testadas perde precisão significativamente. O ataque também exige cooperação entre dois processos maliciosos, trojan e spy, o que caracteriza um canal encoberto e restringe seu uso a cenários de vazamento interno. Além disso, a criação de memória compartilhada via KSM leva dezenas de segundos, e os testes foram feitos em um único modelo de processador.

Em “*Leaking Information Through Cache LRU States*”[Xiong and Szefer 2020] introduz-se a exploração da LRU, revelando falhas em arquiteturas que eram projetadas para isolar acessos. Porém, como qualquer acesso ao conjunto de cache altera o estado LRU, o receptor só pode medir o conjunto uma vez por rodada, obtendo no máximo um bit por medição, o que é uma limitação estrutural do canal. A política PLRU agrava isso,

gerando erros de leitura. A técnica de medição por *pointer chasing* também exige calibrar o tamanho da lista ligada. Por fim, a taxa de transmissão varia bastante conforme o cenário (600 Kbps em *hyper-threading* contra poucos bps em compartilhamento por tempo).

Esses trabalhos adotam metodologias bem distintas entre si. [Liu et al. 2015] avalia ataques na LLC compartilhada, implicando em modelos de ameaça onde atacar a LLC permite ataques entre VMs distintas, enquanto [Xiong and Szefer 2020] concentra-se no estado LRU do cache L1 privado, o que normalmente exige coexecução no mesmo núcleo. Quanto à técnica de medição, [Liu et al. 2015] usam *Prime+Probe*, [Xiong and Szefer 2020] usam uma variação de *pointer chasing* e [Yao et al. 2018] medem diretamente a latência de acesso a blocos em diferentes estados de coerência.

3.2. Mitigações e Lacunas da Literatura

No escopo das defesas, “*CATalyst: Defeating last-level cache side channel attacks in cloud computing*” [Liu et al. 2016] propõe o uso da tecnologia Cache Allocation Technology (CAT) do hardware para particionar a LLC e neutralizar ataques. O artigo dialoga com essa solução ao tratar a eficácia de mecanismos de isolamento, haja visto que as defesas puramente baseadas em software ou particionamento parcial podem ser contornadas. A limitação é que a solução depende da tecnologia Intel CAT, herdando o limite de apenas 4 *Classes of Service*, insuficiente para isolar muitas VMs. O particionamento fino por página também tem limites próprios, como poucas páginas seguras por VM e perda de desempenho. Além disso, o CATalyst neutraliza ataques por contenção de linha, mas não os canais baseados em metadados de coerência ou de substituição LRU explorados em [Yao et al. 2018] e [Xiong and Szefer 2020].

Os ambientes experimentais das metodologias também revelam lacunas: trabalhos como [Liu et al. 2015] e [Liu et al. 2016] testam em máquinas virtualizadas simulando nuvem, enquanto [Yao et al. 2018] e [Xiong and Szefer 2020] testam em processos nativos sem virtualização.

Desta forma, embora Xiong et al. [Xiong et al. 2021] tenham demonstrado a viabilidade de canais encobertos baseados no estado de substituição LRU, a avaliação de contramedidas na literatura carece de uma base comparativa quantitativa e unificada. A pesquisa original foca sua avaliação empírica via simulação (gem5) apenas nas defesas de *Partition-Locked (PL) cache* e *Random Fill (RF) cache*. Outras abordagens estado-da-arte, como DAWG, ScatterCache, CEASER e InvisiSpec, tiveram sua resiliência analisada de forma estritamente qualitativa. Há uma lacuna significativa na literatura quanto à medição empírica da banda residual do canal encoberto frente a essas defesas.

Além disso, estudos anteriores assumem que canais baseados em LRU em níveis mais baixos da hierarquia de memória (L2/LLC) seriam facilmente mitigáveis por técnicas preexistentes em L1. Tal suposição é feita sem uma validação experimental rigorosa e desconsidera o comportamento de políticas de substituição modernas comumente empregadas na LLC, como RRIP e suas variantes.

Para preencher essas lacunas dentro de um escopo viável, este trabalho propõe uma avaliação experimental focada, utilizando o simulador gem5 em modo *syscall-emulation* (SE). Nossa metodologia adapta os protocolos de canais com e sem memória compartilhada (Algoritmos 1 e 2 de [Xiong et al. 2021]) para medir métricas de segurança em conjunto com métricas de desempenho.

Em suma, este artigo apresenta as seguintes contribuições principais:

- **Validação Empírica na LLC:** A avaliação da capacidade do canal de substituição contra políticas modernas na LLC (como a família RRIP, nativa ou adaptada no ecossistema gem5), testando a suposição teórica da literatura de que camadas mais baixas da hierarquia seriam inerentemente mais fáceis de proteger;
- **Análise de Desempenho Integrada:** Uma avaliação do *trade-off* entre segurança e desempenho, medindo o impacto em IPC sob cargas de trabalho abertas e controladas (ex.: PARSEC, MiBench ou microbenchmarks sintéticos);
- **Extensão Comparativa (Opcional):** Caso a curva de aprendizado da ferramenta permita, a implementação e medição quantitativa de uma defesa por particionamento dinâmico do estado de substituição (ex.: adaptação do conceito do DAWG) para fins de comparação direta com os dados originais.

4. Metodologia

5. Uso de IA

[...] falar do uso para a simulação do gem5 e onde usou

References

- [Jaleel et al. 2010] Jaleel, A., Theobald, K. B., Steely, S. C., and Emer, J. (2010). High performance cache replacement using re-reference interval prediction (rrip). In *Proceedings of the 37th Annual International Symposium on Computer Architecture, ISCA '10*, page 60–71, New York, NY, USA. Association for Computing Machinery.
- [Liu et al. 2016] Liu, F., Ge, Q., Yarom, Y., Mckeen, F., Rozas, C., Heiser, G., and Lee, R. B. (2016). Catalyst: Defeating last-level cache side channel attacks in cloud computing. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 406–418.
- [Liu et al. 2015] Liu, F., Yarom, Y., Ge, Q., Heiser, G., and Lee, R. B. (2015). Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy*, pages 605–622.
- [Xiong et al. 2021] Xiong, W., Katzenbeisser, S., and Szefer, J. (2021). Leaking information through cache lru states in commercial processors and secure caches. *IEEE Transactions on Computers*, 70(4):511–523.
- [Xiong and Szefer 2020] Xiong, W. and Szefer, J. (2020). Leaking information through cache lru states. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 139–152.
- [Yao et al. 2018] Yao, F., Doroslovacki, M., and Venkataramani, G. (2018). Are coherence protocol states vulnerable to information leakage? In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 168–179.